Automated Backup System using AWS Lambda

Prepared in the partial fulfillment of the Summer Internship Program on

AWS Cloud Computing with DevOps

AT

Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

DENDUKURI SRIDEVI, 23551A4421, GIET EAST GODAVARI, (Batch – 1)

KANCHARLAPALLI DEVI SRI LAYA, 23551A4441, GIET EAST GODAVARI, (Batch – 2)

YARRAMSETTI JYOTHI SRI, 23551A44B8, GIET EAST GODAVARI, (Batch – 2)

MOTURI SAMEERA, 23551A4472, GIET EAST GODAVARI, (Batch – 2)

DEVARA SASIKALA, 23551A0441, GIET EAST GODAVARI, (Batch – 1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Juttuka Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Dendukuri Sridevi, 23551A4421, sridevidendukuri1729@gmail.com.

Kancharlapalli Devi Sri Laya, 23551A4441, kdslaya54@gmail.com.

Yarramsetti Jyothi Sri, 23551A44B8, jyothisriyarramsetti19@gmail.com.

Moturi Sameera, 23551A4472, sameeramoturi27@gmail.com.

Devara Sasikala, 23551A0441, devarasasikala407@gmail.com.

ABSTRACT

This project presents the implementation of an **Automated Backup System using AWS Lambda** to automate the process of creating and managing backups of Amazon Elastic Block Store (EBS) volumes in the Amazon Web Services (AWS) cloud environment. The primary goal is to leverage **serverless computing** to build a secure, scalable, and event-driven backup solution that eliminates manual intervention while ensuring data protection, high availability, and disaster recovery.

The system workflow begins with the creation of an **Amazon EC2 instance** attached to an **Amazon EBS volume**. The target EBS volume is identified using a predefined **Backup** tag. An **AWS Lambda function** developed in Python is automatically triggered to scan the tagged EBS volumes and create snapshots with appropriate descriptions and metadata. The Lambda function also tags each snapshot for easy identification and implements an automated retention policy by deleting snapshots that exceed the configured retention period. The function is granted the required permissions through **AWS Identity and Access Management (IAM)** and is executed automatically at scheduled intervals using **Amazon EventBridge Scheduler**. During execution, all backup activities are recorded in **Amazon CloudWatch Logs**, enabling administrators to monitor the backup process and troubleshoot issues efficiently.

This architecture is fully **serverless**, utilizing **AWS Lambda for automation, Amazon EC2 and Amazon EBS for computing and storage, EventBridge Scheduler for scheduled execution, IAM for secure access control, and CloudWatch for logging and monitoring**. The solution provides reliable, scalable, and cost-effective backup management without requiring dedicated backup servers or manual execution.

By automating snapshot creation and retention management, the system ensures continuous protection of critical cloud data, reduces the risk of data loss caused by accidental deletion or infrastructure failures, and improves operational efficiency. The project demonstrates how AWS serverless services can be effectively integrated to build an enterprise-grade automated backup solution, providing a strong foundation for future enhancements such as **cross-region backup replication, Amazon SNS email notifications, AWS Backup integration, lifecycle policies for long-term storage optimization, and centralized monitoring dashboards**.

TABLE OF CONTENTS

S. No Content **Pg. No**

1. INTRODUCTION

In today's cloud-driven digital landscape, ensuring the availability, integrity, and security of critical business data is a top priority for organizations of all sizes. As businesses increasingly rely on cloud infrastructure to host applications and store valuable information, maintaining regular backups has become essential for preventing data loss caused by hardware failures, accidental deletion, cyberattacks, or system outages. This project, titled **Automated Backup System using AWS Lambda**, leverages the power of **Amazon Web Services (AWS)** serverless computing to deliver a secure, scalable, and fully automated backup solution. It demonstrates how modern cloud technologies can replace traditional manual backup procedures with an intelligent, event-driven architecture that requires minimal infrastructure management.

Modern IT environments demand reliable backup mechanisms that can operate without continuous human intervention while ensuring data recovery whenever required. Conventional backup solutions often require dedicated servers, complex scheduling software, and ongoing maintenance, leading to increased operational costs and administrative overhead. This project addresses these challenges by adopting a **serverless architecture**, where AWS services work together to automate the complete backup lifecycle, including snapshot creation, backup retention, scheduling, and monitoring.

Designed with automation, reliability, and cost optimization as its primary objectives, the system enables administrators to identify Amazon Elastic Block Store (EBS) volumes that require backup through predefined AWS tags. An **AWS Lambda** function, developed in Python using the **Boto3 SDK**, automatically scans the tagged EBS volumes and creates snapshots with descriptive metadata. The Lambda function also applies custom tags to each snapshot, making backup management easier and more organized. To optimize storage utilization and reduce unnecessary costs, the system automatically deletes snapshots that exceed the configured retention period.

One of the key strengths of this project is its ability to perform backup operations without requiring dedicated backup servers or manual execution. The Lambda function is securely authorized using **AWS Identity and Access Management (IAM)** policies and roles, ensuring that only authorized actions are performed on AWS resources. The backup process is automatically triggered at scheduled intervals using **Amazon EventBridge Scheduler**,

enabling regular backups without administrator involvement. During execution, all backup activities, including successful snapshot creation, deletion of expired backups, and error handling, are recorded in **Amazon CloudWatch Logs**, providing complete visibility into system performance and simplifying troubleshooting.

This report presents a comprehensive overview of the Automated Backup System, covering its system architecture, design methodology, implementation process, and integration of AWS services. The project showcases how **AWS Lambda, Amazon EC2, Amazon EBS, EventBridge Scheduler, IAM, and CloudWatch** can be seamlessly integrated to develop a secure, reliable, and enterprise-grade automated backup solution. The modular and scalable design also allows for future enhancements such as **cross-region snapshot replication for disaster recovery, Amazon SNS email notifications, AWS Backup integration, automated lifecycle management policies, and centralized monitoring dashboards**.

By implementing a fully automated, event-driven backup solution, this project significantly improves data protection, reduces operational complexity, minimizes storage costs, and enhances disaster recovery capabilities. It demonstrates the effectiveness of AWS serverless technologies in building cloud-native backup systems capable of meeting modern enterprise requirements while providing a strong foundation for future cloud infrastructure automation.

2. METHODOLOGY

The development of the **Automated Backup System using AWS Lambda** followed a structured and systematic methodology to ensure the successful implementation of an automated, secure, and scalable cloud backup solution. The methodology was divided into several phases, including requirements gathering, system design, implementation, testing, deployment, and iterative refinement. Each phase followed cloud-native development principles and AWS serverless best practices to achieve reliable backup automation. The following subsections describe each phase in detail.

2.1. Requirements Gathering

The project began with an analysis of the functional and technical requirements for an automated cloud backup system. Discussions with mentors and research on cloud backup

strategies helped define the primary objectives: automatically identify EBS volumes that require backup, create snapshots without manual intervention, manage backup retention efficiently, and schedule backups at regular intervals. The system was designed to securely perform snapshot creation, automatically delete expired backups, and provide execution logs for monitoring and troubleshooting.

2.2. Design

Based on the identified requirements, a **serverless, event-driven architecture** was designed using AWS services. The workflow was planned to include **Amazon EC2** and **Amazon EBS** as the storage resources, **AWS Lambda** for backup automation, **Amazon EventBridge Scheduler** for periodic execution, **AWS IAM** for secure permissions, and **Amazon CloudWatch** for logging and monitoring. The architecture was designed to ensure scalability, security, automation, and minimal operational overhead while following AWS best practices.

2.3. Implementation

During the implementation phase, an AWS Lambda function was developed using **Python** and the **Boto3 SDK**. The function scans Amazon EBS volumes for a predefined **Backup=true** tag to identify the resources that require backup. Once identified, the Lambda function automatically creates snapshots with descriptive metadata and applies custom tags for easy identification and lifecycle management. The function also implements an automated retention policy by identifying snapshots older than the configured retention period and deleting them to optimize storage utilization. Appropriate IAM roles and policies were configured to allow the Lambda function to securely access EC2 resources and CloudWatch Logs.

2.4. Backup Management

A snapshot-based backup strategy was adopted using **Amazon EBS Snapshots**, providing incremental, durable, and highly available backups. Each snapshot is tagged with metadata such as the source volume ID and creator information to simplify backup management. The automated retention mechanism ensures that only recent snapshots are retained while obsolete backups are removed automatically. This approach minimizes storage costs while maintaining an effective disaster recovery strategy.

2.5. Testing

Several levels of testing were performed to validate the functionality and reliability of the system. Unit testing was conducted on the Lambda function to verify snapshot creation, tag validation, and retention logic. Integration testing ensured seamless interaction among AWS services including EC2, EBS, Lambda, EventBridge Scheduler, IAM, and CloudWatch. Functional testing involved manually triggering the Lambda function and verifying successful snapshot creation, automatic deletion of expired snapshots, and accurate logging of execution details within CloudWatch.

2.6. Deployment

The complete solution was deployed using the **AWS Management Console**. IAM roles and policies were configured according to the principle of least privilege to ensure secure access between AWS services. The Lambda function was deployed with the required runtime configuration and an increased execution timeout to support backup operations. **Amazon EventBridge Scheduler** was configured using a cron expression to automatically execute the Lambda function at predefined intervals. **Amazon CloudWatch Logs** was enabled to continuously monitor execution status, system performance, and operational errors.

2.7. Iterative Refinement

Throughout the project development, continuous testing and feedback were used to improve system reliability and performance. Modifications were made to enhance error handling, improve snapshot tagging, optimize retention logic, and strengthen security through refined IAM permissions. The system architecture was periodically reviewed to ensure compliance with AWS serverless best practices and maintain scalability. The modular design also enables future enhancements such as **cross-region snapshot replication, Amazon SNS email notifications, AWS Backup integration, automated lifecycle policies, and centralized CloudWatch dashboards**.

The methodology adopted for the development of the **Automated Backup System using AWS Lambda** ensured the successful implementation of a **secure, scalable, automated, and cost-effective cloud backup solution**. By following a serverless and event-driven approach, the project demonstrates the practical application of AWS cloud technologies in automating backup operations, improving disaster recovery, reducing administrative effort, and enhancing the reliability of cloud infrastructure.

3. TECHNOLOGY STACK

The **Automated Backup System using AWS Lambda** was developed using a modern, cloud-native technology stack that supports serverless computing, event-driven automation, secure cloud resource management, and scalable backup operations. The following technologies and AWS services were used throughout the implementation:

3.1. Cloud Platform

- **Amazon Web Services (AWS):** The entire project is deployed on AWS, utilizing its serverless services to automate EBS volume backups, schedule executions, manage permissions, and monitor system performance.

3.2. AWS Services

- **Amazon EC2 (Elastic Compute Cloud):** Hosts the virtual machine instances whose attached EBS volumes are backed up by the system.
- **Amazon EBS (Elastic Block Store):** Provides persistent block storage volumes attached to EC2 instances. The system creates snapshots of these volumes for backup and disaster recovery.
- **AWS Lambda:** Executes the core backup logic by identifying tagged EBS volumes, creating snapshots, applying metadata tags, and deleting expired snapshots based on the configured retention policy.
- **Amazon EventBridge Scheduler:** Automatically triggers the Lambda function at predefined intervals using cron expressions, enabling fully automated scheduled backups.
- **AWS IAM (Identity and Access Management):** Provides secure access control by defining roles and permissions that allow the Lambda function to interact safely with EC2, EBS, and CloudWatch services.
- **Amazon CloudWatch:** Monitors Lambda execution, records system logs, and provides operational insights for debugging, performance analysis, and troubleshooting.

3.3. Programming & Scripting

- **Python 3.12 (AWS Lambda Runtime):** Used to develop the serverless backup application running inside AWS Lambda.
- **Boto3 SDK:** The official Python SDK for AWS used within the Lambda function to interact with Amazon EC2, Amazon EBS, CloudWatch Logs, and other AWS services.

3.4. Backup Mechanism

- **Amazon EBS Snapshots:** Used to create incremental backups of EBS volumes. Snapshots are securely stored within AWS and support efficient data recovery and disaster recovery operations.
- **AWS Resource Tags:** Used to identify EBS volumes eligible for backup (for example, **Backup = true**) and to manage snapshots created by the automated system.

3.5. Security & Access

- **IAM Roles and Policies:** Configured following the principle of least privilege to grant the Lambda function only the permissions required to create, delete, describe, and tag EBS snapshots, along with access to CloudWatch Logs.

3.6. Monitoring & Logging

- **CloudWatch Logs:** Records detailed execution logs generated by the Lambda function, including snapshot creation status, deletion of expired snapshots, and error messages for troubleshooting.
- **CloudWatch Metrics:** Provides real-time monitoring of Lambda invocations, execution duration, success rate, failures, and overall system performance.

The combination of these AWS services and technologies enables the **Automated Backup System using AWS Lambda** to provide a **secure, scalable, reliable, and fully automated cloud backup solution**. The serverless architecture minimizes infrastructure management while ensuring efficient backup scheduling, monitoring, and disaster recovery for Amazon EC2 and EBS resources.

4. SYSTEM DESIGN / ARCHITECTURE

The **Automated AWS EBS Backup System** is designed as a **serverless, event-driven cloud solution** that automates the creation and management of Amazon Elastic Block Store (EBS) snapshots. The architecture eliminates manual backup operations by using AWS managed services, ensuring reliability, scalability, cost optimization, and enhanced data protection. The system follows a serverless architecture consisting of resource identification, backup processing, snapshot management, scheduling, and security layers. This section describes the major components and their interactions.

4.1. Resource Identification Layer – Amazon EC2 (EBS Volumes)

The backup process begins by identifying the Amazon EBS volumes that require backup. Instead of backing up every volume, only volumes tagged with a predefined tag are selected.

Tag Configuration

- **Key:** Backup
- **Value:** true

The AWS Lambda function scans all EBS volumes and filters only those containing this tag. This tag-based approach allows administrators to control backup policies without modifying application code.

Figure 2. Tagged EBS Volume (Backup = true)

Key Features

- Tag-based volume selection
- Supports multiple EBS volumes
- Easy backup policy management
- No manual volume selection required

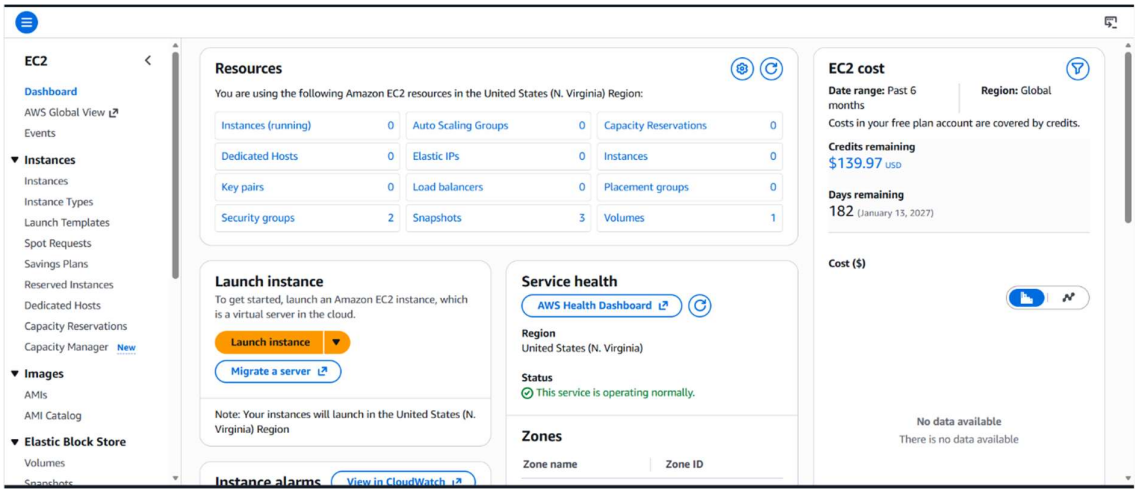


Figure 1. Amazon EC2 Dashboard

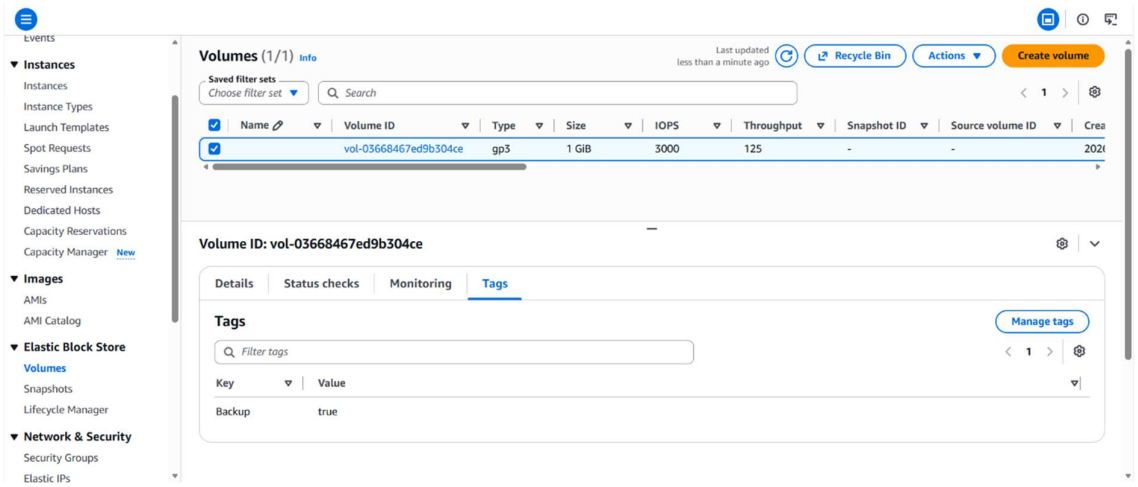


Figure 2. Tagged EBS Volume (Backup = true)

4.2. Backup Processing Layer – AWS Lambda

An AWS Lambda function named **AutomatedEBSBackup** acts as the processing engine of the system. The function executes either manually or automatically through Amazon EventBridge Scheduler.

The Lambda function performs the following operations:

- Retrieves all tagged EBS volumes
- Creates snapshots for every eligible volume

- Generates descriptive snapshot names
- Adds management tags to snapshots
- Logs execution details in Amazon CloudWatch
- Removes snapshots older than the configured retention period

The backup retention period is configured inside the Lambda function.

Retention Period: 7 Days

Processing Workflow

1. Read tagged EBS volumes
2. Create snapshot
3. Tag newly created snapshot
4. Scan previous snapshots
5. Delete expired snapshots
6. Return backup summary

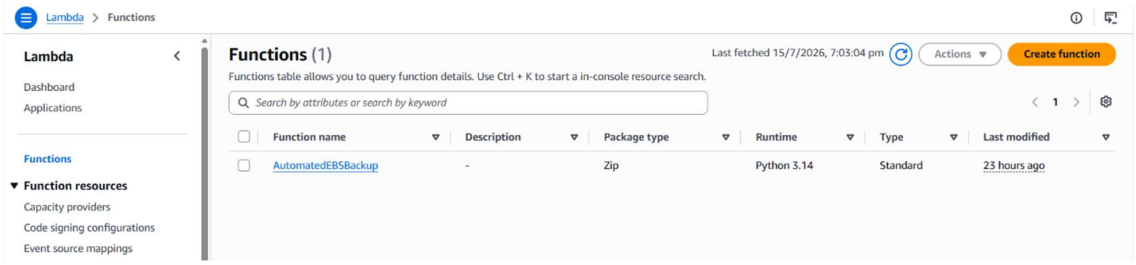


Figure 3. Lambda Function (AutomatedEBSBackup)

Figure 4. Lambda Function Code

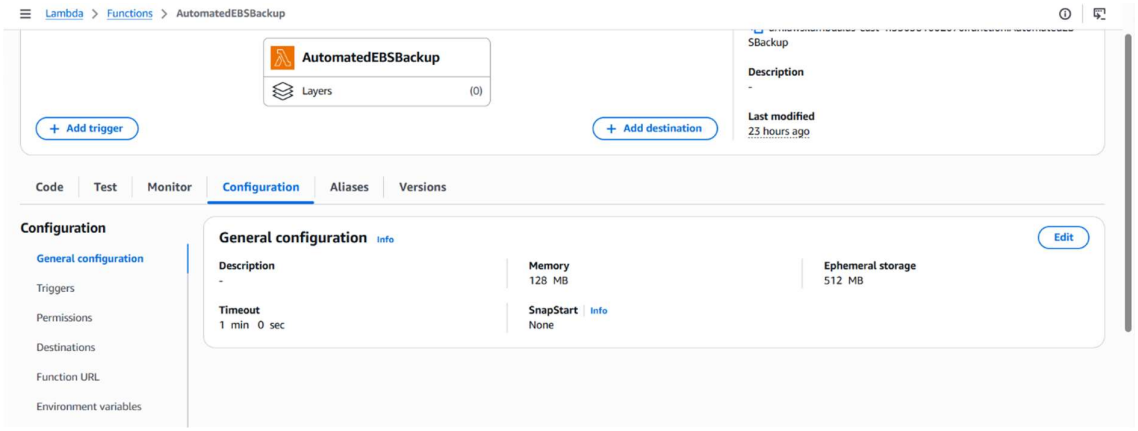


Figure 5. Lambda Configuration Settings

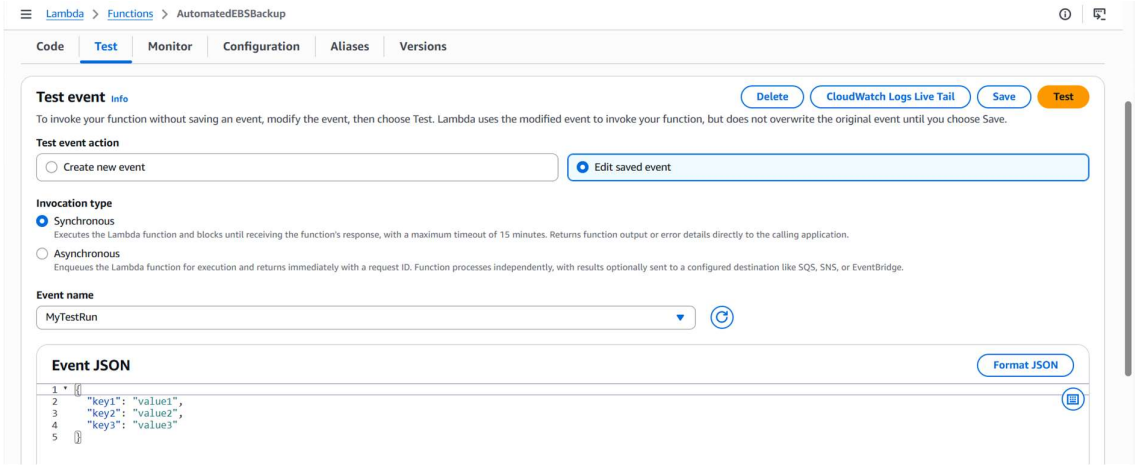


Figure 6. Lambda Test Event Configuration

4.3. Snapshot Storage Layer – Amazon EBS Snapshots

Each successful backup is stored as an Amazon EBS Snapshot. Snapshots are incremental backups managed by Amazon EC2.

Every snapshot contains metadata including:

- Snapshot ID
- Volume ID
- Creation Date
- Description
- CreatedBy Tag
- VolumeId Tag

Snapshots are retained only for the configured number of days, helping reduce storage costs.

Features

- Incremental storage
- Durable backup storage
- Automatic snapshot tagging
- Cost-efficient backup retention

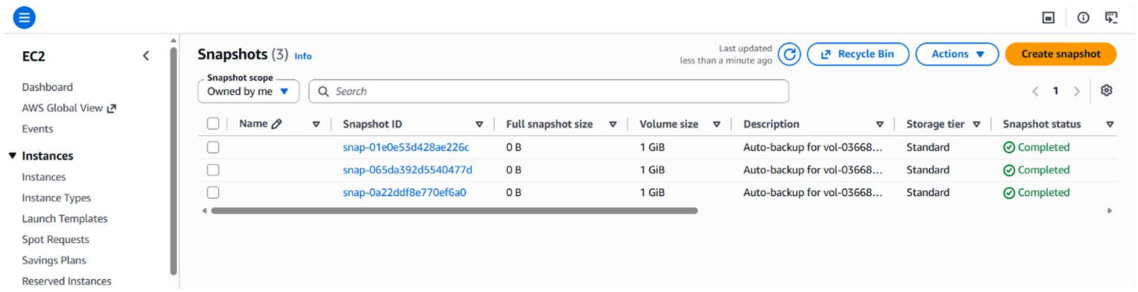


Figure 7. Amazon EC2 Snapshot List

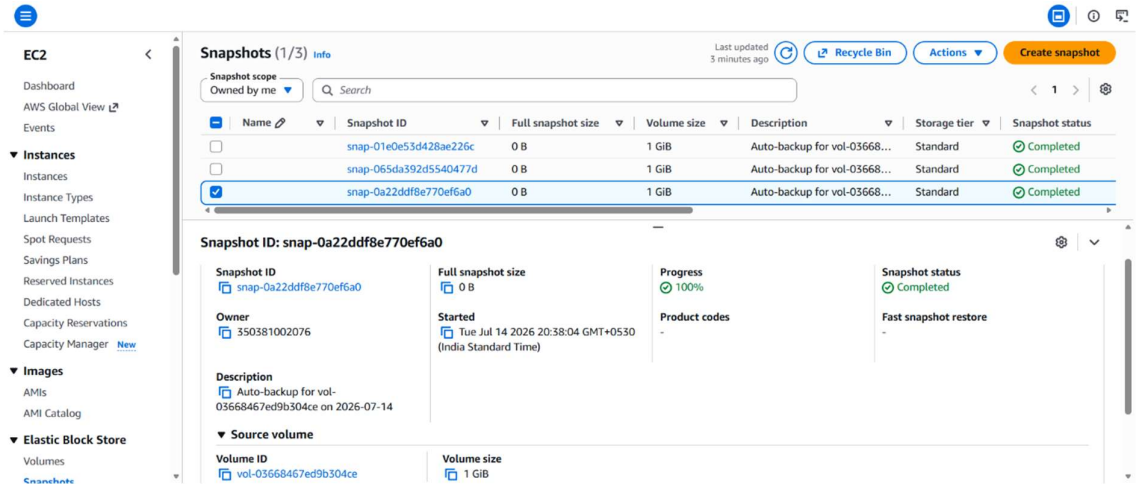


Figure 8. Snapshot Details

4.4. Automation Layer – Amazon EventBridge Scheduler

The entire backup process is automated using Amazon EventBridge Scheduler.

A recurring cron schedule triggers the Lambda function every day without requiring user intervention.

Cron Expression

cron(0 0 * * ? *)

This schedule runs every day at **12:00 AM UTC**.

Whenever the scheduled time arrives:

- EventBridge invokes the Lambda function
- Lambda creates new snapshots
- Old snapshots are removed automatically

Features

- Fully automated execution
- Cron-based scheduling
- Reliable serverless orchestration

- No server management required

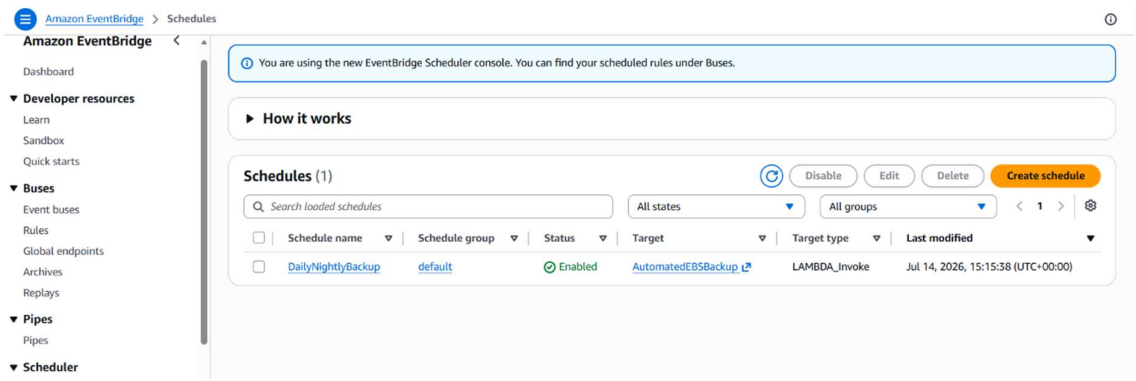


Figure 9. EventBridge Scheduler

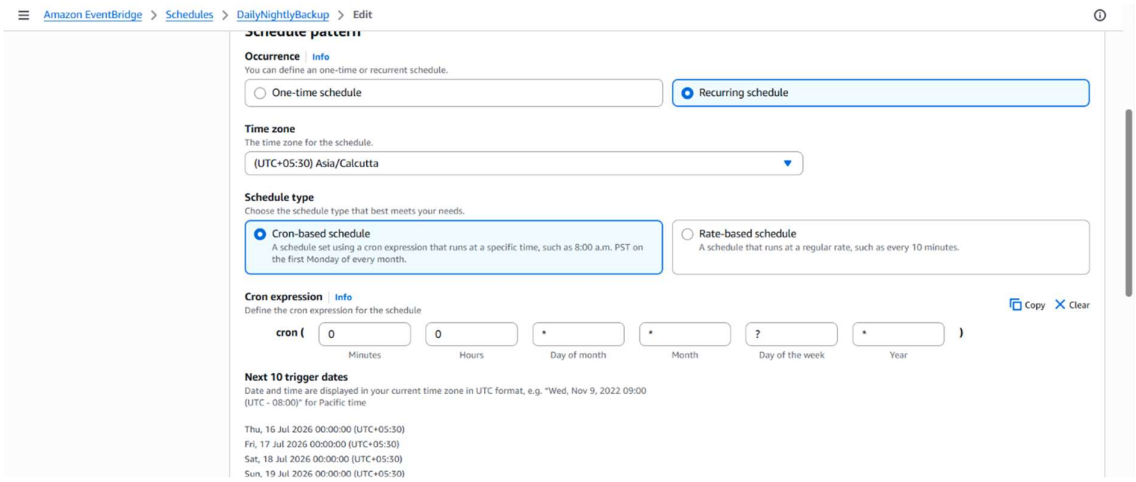


Figure 10. Schedule Configuration

4.5. Monitoring Layer – Amazon CloudWatch

Amazon CloudWatch automatically records execution logs generated by the Lambda function.

CloudWatch stores:

- Backup execution status
- Number of snapshots created
- Number of deleted snapshots

- Execution errors
- Debug messages

Administrators can monitor backup health and troubleshoot issues directly from CloudWatch Logs.

Features

- Real-time monitoring
- Automatic log collection
- Error tracking
- Performance analysis

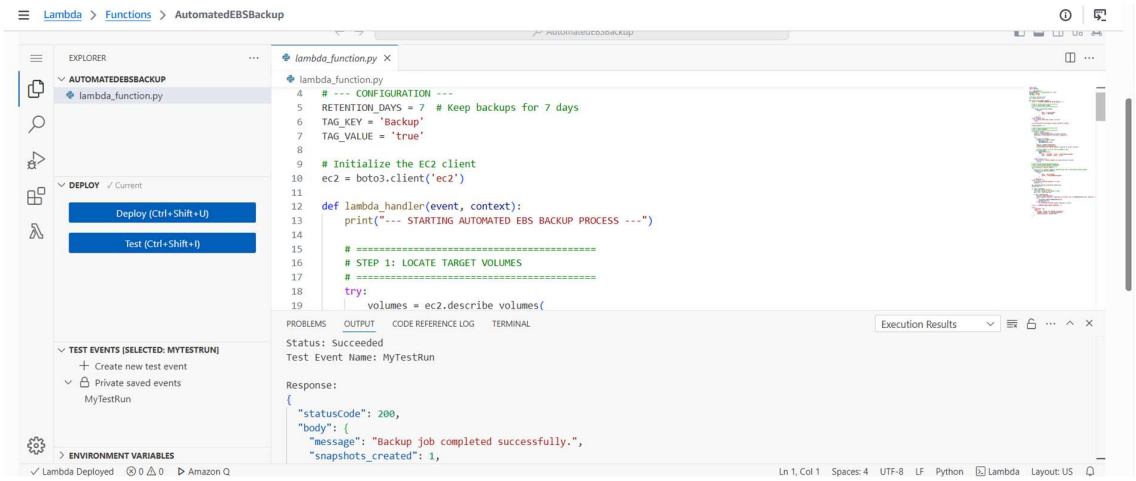


Figure 11. Lambda Execution Logs

4.6. Security and Access Control – AWS IAM

AWS Identity and Access Management (IAM) provides secure access between AWS services.

A dedicated IAM Role named **LambdaBackupExecutionRole** is attached to the Lambda function.

The role uses the custom policy **LambdaEBSBackupPolicy**, which grants only the permissions required for backup operations.

IAM Permissions

- Create EBS Snapshots
- Delete EBS Snapshots
- Describe Volumes
- Describe Snapshots
- Create Snapshot Tags
- Create CloudWatch Logs
- Write CloudWatch Logs

The system follows the **Principle of Least Privilege**, ensuring the Lambda function receives only the minimum permissions necessary.

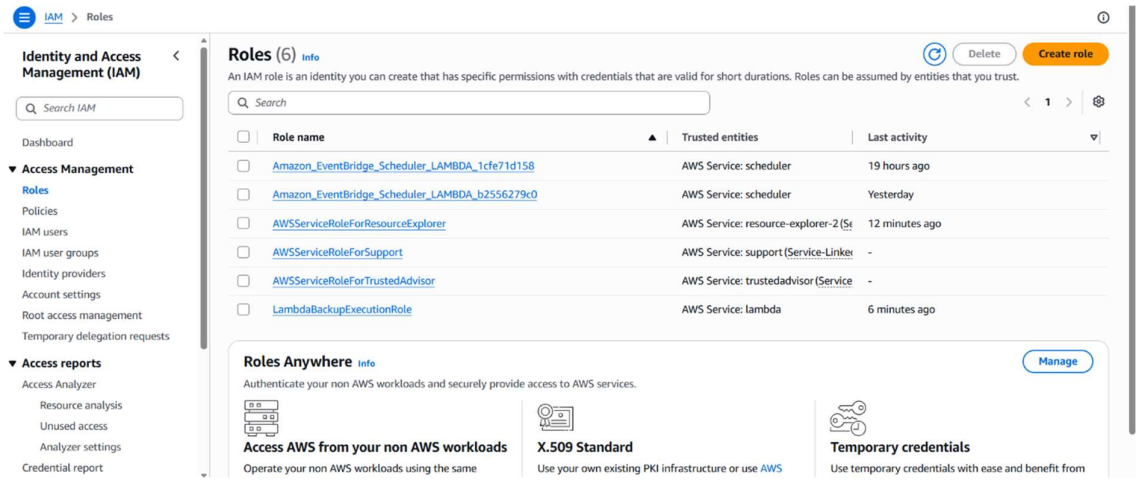


Figure 12. IAM Role (LambdaBackupExecutionRole)

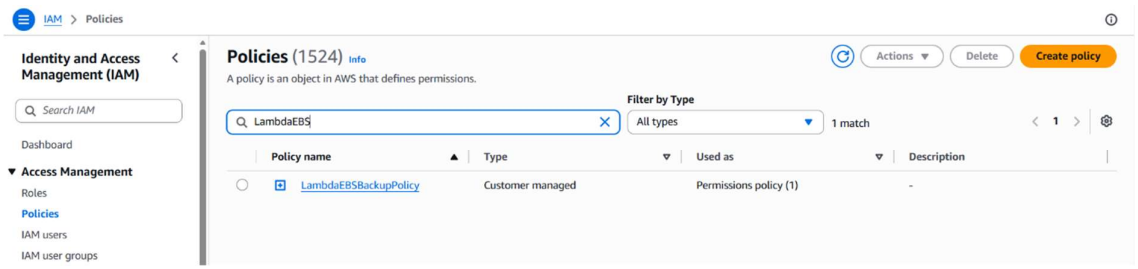


Figure 13. LambdaEBSBackupPolicy

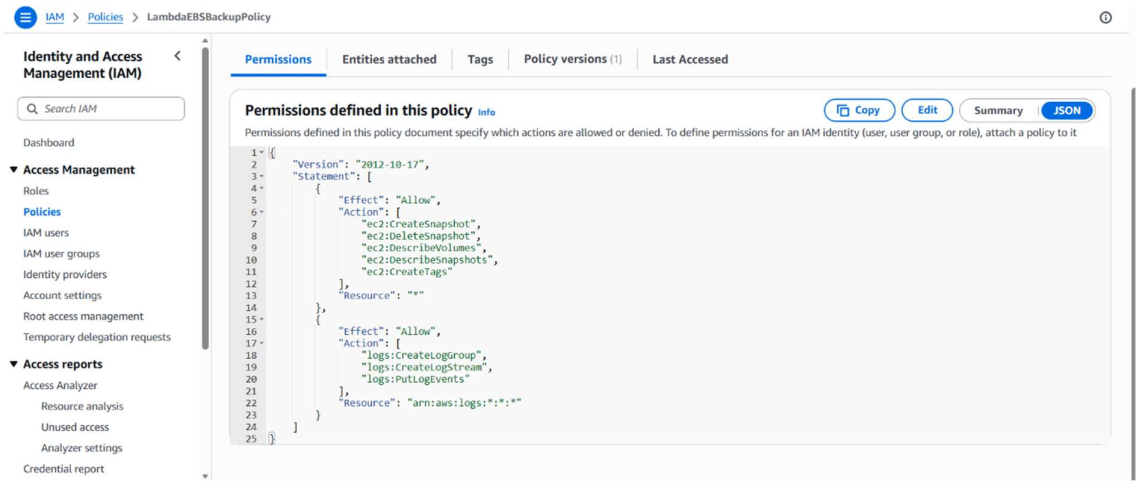


Figure 14. IAM Policy JSON

4.7. Overall System Workflow

The complete backup process follows these steps:

1. Administrator tags an EBS volume with **Backup = true**.
2. Amazon EventBridge triggers the **AutomatedEBSBackup** Lambda function based on the configured schedule.
3. Lambda identifies all tagged EBS volumes.
4. Snapshots are created for each eligible volume.
5. Newly created snapshots are tagged for identification.
6. Lambda scans all snapshots created by the backup system.
7. Snapshots older than **7 days** are automatically deleted.
8. CloudWatch records execution logs for monitoring and troubleshooting.

This architecture provides a **fully automated, serverless, secure, scalable, and cost-effective backup solution** using AWS managed services. By integrating **Amazon EC2, AWS Lambda, Amazon EventBridge, Amazon EBS Snapshots, CloudWatch, and IAM**, the system ensures reliable data protection while minimizing operational effort and storage costs.

5. IMPLEMENTATION

The implementation of the **Automated AWS EBS Backup System** converts the proposed architecture into a fully functional, serverless backup solution. The system automates the process of identifying Amazon EBS volumes, creating snapshots, managing backup retention, and scheduling backup operations using AWS managed services. By integrating **AWS Lambda, Amazon EC2, Amazon EventBridge, IAM, and Amazon CloudWatch**, the solution provides secure, reliable, and cost-effective backup automation without requiring manual intervention or dedicated servers.

5.1. Creating IAM Policy and Execution Role

The first step in the implementation is configuring the necessary permissions for the Lambda function.

A custom IAM policy named **LambdaEBSBackupPolicy** is created to allow the Lambda function to perform backup operations.

The policy grants permission to:

- Create EBS snapshots
- Delete old snapshots
- Describe EBS volumes
- Describe snapshots
- Create tags on snapshots
- Create and write CloudWatch logs

An IAM Role named **LambdaBackupExecutionRole** is then created and attached to the Lambda function.

This role follows the **Principle of Least Privilege**, ensuring that only the required permissions are granted.

Key IAM Details

- **Policy Name:** LambdaEBSBackupPolicy

- **Role Name:** LambdaBackupExecutionRole
- **Trusted Service:** AWS Lambda

5.2. AWS Lambda for Automated Backup Processing

The core functionality of the system is implemented using an AWS Lambda function written in **Python 3.12**.

The Lambda function is responsible for automating the entire backup process.

It performs the following operations:

1. Retrieves all EBS volumes tagged with **Backup = true**.
2. Creates snapshots for each eligible volume.
3. Generates descriptive snapshot names.
4. Adds management tags to every snapshot.
5. Retrieves previously created snapshots.
6. Deletes snapshots older than the configured retention period (7 days).
7. Returns the execution summary.

Key Lambda Details

- Runtime: **Python 3.12**
- Function Name: **AutomatedEBSBackup**
- Execution Role: **LambdaBackupExecutionRole**
- Timeout: **1 Minute**
- Retention Period: **7 Days**

Logging is automatically enabled through **Amazon CloudWatch Logs** for monitoring and troubleshooting.

5.3. Backup Storage Using Amazon EBS Snapshots

Whenever the Lambda function is executed, it creates an incremental snapshot of every tagged EBS volume.

Each snapshot contains important metadata such as:

- Snapshot ID
- Volume ID
- Creation Date
- Description
- CreatedBy Tag
- VolumeId Tag

Snapshots provide reliable and durable backups while minimizing storage costs because Amazon stores only changed blocks after the first snapshot.

Snapshot Features

- Incremental backup storage
- Automatic snapshot tagging
- Secure backup storage
- Fast recovery capability
- Cost-efficient backup management

5.4. Automated Scheduling Using Amazon EventBridge

To eliminate manual execution, the Lambda function is scheduled using **Amazon EventBridge Scheduler**.

A recurring **Cron Expression** automatically invokes the backup function every day.

Schedule Details

- **Schedule Name:** DailyNightlyBackup
- **Schedule Type:** Recurring
- **Cron Expression:** cron(0 0 * * ? *)
- **Execution Time:** Every day at 12:00 AM UTC

Once the schedule is created, backups occur automatically without user interaction.

5.5. Snapshot Retention and Cleanup

To optimize storage costs, the system automatically removes outdated snapshots.

The Lambda function scans all snapshots created by the backup system using the **CreatedBy** tag.

If the snapshot age exceeds the configured retention period (**7 days**), it is automatically deleted.

This cleanup mechanism ensures:

- Reduced storage costs
- Automatic lifecycle management
- Elimination of unnecessary backups
- Efficient snapshot organization

5.6. Monitoring with Amazon CloudWatch

Amazon CloudWatch is used to monitor the execution of the Lambda function.

CloudWatch automatically records:

- Lambda execution logs
- Number of snapshots created
- Number of deleted snapshots
- Execution duration
- Errors and exceptions
- Debug messages

CloudWatch enables administrators to monitor backup activities and quickly identify any failures.

Optional CloudWatch alarms can also be configured to notify administrators if backup execution fails.

5.7. Security and Access Control

Security is implemented using **AWS Identity and Access Management (IAM)**.

The Lambda function is granted access only to the AWS resources required for backup operations.

The system uses:

- **IAM Role:** LambdaBackupExecutionRole
- **IAM Policy:** LambdaEBSBackupPolicy

The policy allows:

- Read access to EBS volume information
- Snapshot creation
- Snapshot deletion
- Snapshot tagging
- CloudWatch log creation

Following the **Principle of Least Privilege** improves the security of the solution by preventing unauthorized access to AWS resources.

5.8. Manual Testing and Validation

Before enabling automation, the Lambda function is tested manually using the built-in **Test Event** feature.

During testing:

- A test event is created.
- The Lambda function is executed.
- Execution results are verified.
- Newly created snapshots are confirmed in the EC2 Snapshot Console.
- CloudWatch logs are checked for successful execution.

A successful execution returns a response similar to:

```json
{
  "statusCode": 200,
  "body": {
    "message": "Backup job completed successfully.",
    "snapshots_created": 1,
    "snapshots_deleted": 0
  }
}
```

This confirms that the backup process is functioning correctly.

The successful implementation of the **Automated AWS EBS Backup System** demonstrates how AWS serverless services can be integrated to create a secure, scalable, and fully automated backup solution. By combining **AWS Lambda, Amazon EC2, Amazon EBS Snapshots, Amazon EventBridge, IAM, and Amazon CloudWatch**, the system minimizes manual effort, reduces operational costs, and ensures reliable data protection. The modular architecture also provides flexibility for future enhancements such as backup notifications using Amazon SNS, multi-region backup replication, lifecycle policies, and disaster recovery automation.

6. RESULTS

The implementation of the **Automated AWS EBS Backup System** successfully achieved the project's objective of automating EBS volume backups using AWS serverless services. The system was tested under multiple scenarios to validate its functionality, reliability, automation, and security. The results demonstrate that the solution efficiently creates scheduled backups, manages snapshot retention, and reduces manual administrative effort while maintaining data protection.

6.1. Automated EBS Snapshot Creation

The Lambda function successfully identified all Amazon EBS volumes tagged with **Backup = true** and automatically created snapshots for each eligible volume.

Each snapshot was generated with:

- Unique Snapshot ID

- Volume ID
- Creation Timestamp
- Backup Description
- Management Tags

The snapshots were successfully stored in Amazon EC2 without requiring manual intervention.

6.2. Automated Snapshot Retention Management

The system successfully implemented the snapshot retention policy.

During execution, the Lambda function:

- Retrieved previously created snapshots.
- Calculated the age of each snapshot.
- Deleted snapshots older than **7 days**.
- Preserved recent snapshots for recovery.

This automatic cleanup reduced unnecessary storage consumption and optimized AWS storage costs.

6.3. Fully Automated Backup Scheduling

Amazon EventBridge Scheduler successfully invoked the Lambda function according to the configured cron schedule.

Testing confirmed that:

- Backups were executed automatically every day.
- No manual execution was required.
- Scheduled events triggered the Lambda function reliably.
- Snapshot creation occurred at the scheduled time.

The automation significantly reduced administrative effort while ensuring consistent backup operations.

6.4. Scalable Serverless Architecture

The solution demonstrated the scalability advantages of AWS serverless computing.

The architecture successfully handled:

- Single EBS volume backup
- Multiple tagged EBS volumes
- Consecutive Lambda executions
- Repeated scheduled backups

Since AWS Lambda automatically scales based on workload, no infrastructure management or capacity planning was required.

Testing scenarios included:

- Backup of one tagged volume
- Backup of multiple tagged volumes
- Manual Lambda invocation
- Scheduled EventBridge execution
- Automatic snapshot cleanup

All test cases completed successfully without performance degradation.

6.5. Secure Service Access Using IAM

AWS Identity and Access Management (IAM) successfully enforced secure communication between AWS services.

The configured IAM Role allowed the Lambda function to:

- Describe EBS volumes
- Create snapshots
- Delete expired snapshots
- Create snapshot tags
- Write execution logs to CloudWatch

No unauthorized operations were permitted, validating that the system follows AWS security best practices using the **Principle of Least Privilege**.

6.6. System Monitoring and Logging

Amazon CloudWatch successfully monitored every execution of the Lambda function.

The logs recorded:

- Backup execution status
- Number of snapshots created
- Number of deleted snapshots
- Execution duration
- Error messages (if any)
- Debug information

CloudWatch logs confirmed successful execution for all test scenarios and provided detailed information for troubleshooting during development.

6.7. Manual Testing Results

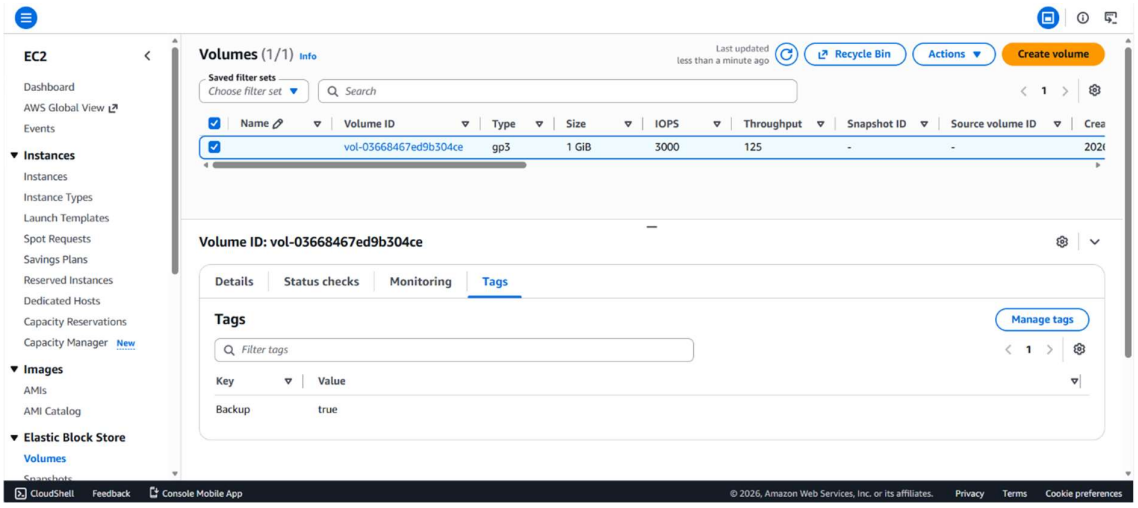
The Lambda function was tested using the built-in **Test Event** feature.

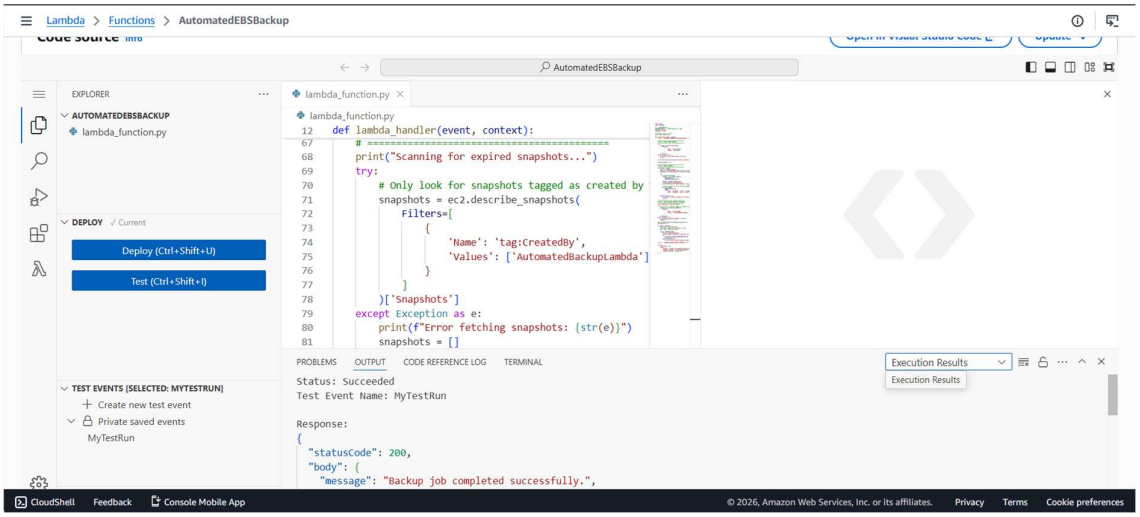
A successful execution returned the following response:

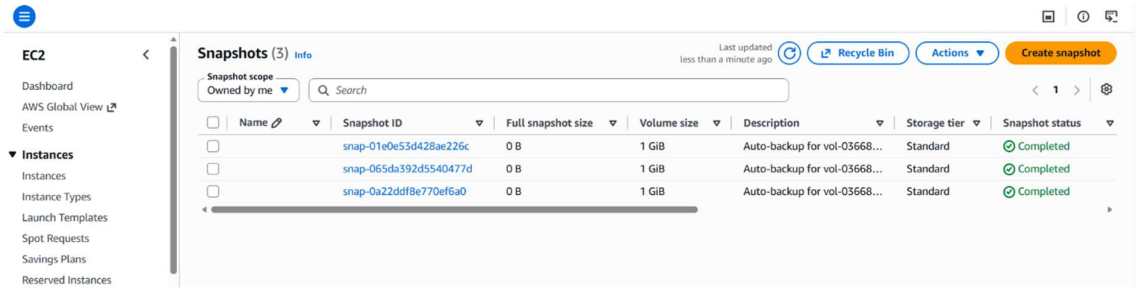
```
JSON

{
  "statusCode": 200,
  "body": {
    "message": "Backup job completed successfully.",
    "snapshots_created": 1,
    "snapshots_deleted": 0
  }
}
```

The corresponding snapshot was immediately visible in the **Amazon EC2 Snapshots Console**, confirming that the backup process executed correctly.

6.7. Screenshots of the Output







7. CONCLUSION

The successful development and deployment of the **Automated AWS EBS Backup System** demonstrate the effectiveness of serverless cloud computing in automating critical backup operations for cloud infrastructure. By leveraging core AWS services such as **Amazon EC2, AWS Lambda, Amazon EventBridge, Amazon EBS Snapshots, AWS IAM, and Amazon CloudWatch**, the system achieves its primary objective of creating automated, reliable, and secure backups of Amazon EBS volumes without requiring manual intervention or dedicated backup servers.

The project highlights the key advantages of a **serverless architecture**, including automatic scaling, reduced operational overhead, cost optimization, and improved reliability. AWS Lambda enables on-demand execution of backup tasks, while Amazon EventBridge automates scheduled execution. Amazon EBS Snapshots provide durable and incremental backup storage, helping minimize storage costs while ensuring data protection. CloudWatch offers centralized monitoring and logging, allowing administrators to track backup execution and troubleshoot issues efficiently.

Throughout the implementation, special emphasis was placed on **security, automation, and resource management**. IAM roles and policies were configured following the **Principle of Least Privilege**, ensuring secure interaction between AWS services. The automated snapshot retention policy effectively manages backup lifecycle by deleting outdated snapshots after the configured retention period, thereby optimizing storage utilization and reducing unnecessary costs.

Comprehensive testing confirmed that the system successfully:

- Automatically identifies tagged EBS volumes.
- Creates snapshots for selected volumes.
- Deletes expired snapshots based on the retention policy.
- Executes scheduled backups using Amazon EventBridge.
- Records execution details through Amazon CloudWatch.
- Operates reliably without manual intervention.

Beyond its current implementation, the project provides a solid foundation for several future enhancements, including:

- Integrating **Amazon SNS** for email or SMS backup notifications.
- Implementing **cross-region snapshot replication** for disaster recovery.
- Developing a **web-based backup monitoring dashboard**.
- Adding **AWS Backup** integration for centralized backup management.
- Supporting automated backup reports and compliance auditing.
- Implementing lifecycle policies for long-term snapshot archiving.
- Enabling backup management through **Amazon API Gateway** and AWS SDKs.

In conclusion, the **Automated AWS EBS Backup System** serves as a practical, secure, and scalable example of serverless infrastructure automation on AWS. The project demonstrates how multiple AWS managed services can be integrated to build a cost-effective, enterprise-grade backup solution that enhances data protection, minimizes administrative effort, and supports business continuity. Its modular architecture and extensibility make it well-suited for production environments and future cloud infrastructure management enhancements.